Documentation – Serveur Web Multi Applications

Zyad LAOUANI, Arthur DEMARCQ, Louis GODARD

Description du projet

Ce projet permet d'héberger plusieurs applications web sur un seul serveur, en utilisant des sous-domaines distincts et en assurant leur accessibilité via HTTPS.

L'objectif est de centraliser le déploiement d'applications développées avec différentes technologies, tout en garantissant sécurité, performance et simplicité d'administration.

Objectifs du projet

Ce projet doit permettre d'héberger plusieurs applications sur un même serveur, d'utiliser un reverse proxy pour gérer les flux, de sécuriser les accès via HTTPS (certificats SSL/TLS), de gérer les sous-domaines pour chaque application et de faciliter l'ajout et la suppression d'applications

Prérequis:

Pour commencer, on a besoin de deux machines virtuelles (une qui va servir de routeur, une en host-only utilisée pour héberger le site web)

Pour l'installation suivez ces étapes

Manipulations pour Linux

Tout d'abord, on va paramétrer le site web avec Nginx sur la machine serveur, on commence par l'installer avec ***sudo apt install nginx***

On va également avoir besoin de PHP car Nginx n'a pas de module intégré pour le gérer.

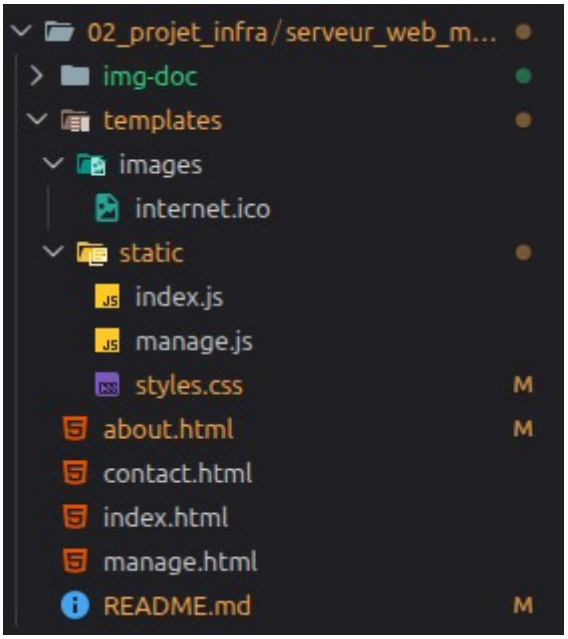
sudo apt install php-fpm php-mysql

Pour plus tard, on va devoir vérifier la version de PHP que l'on, on fait alors
ls /var/run/php/

```
root@vm-roueur1:/var/www/html# ls /var/run/php/
php8.4-fpm.pid  php8.4-fpm.sock  php-fpm.sock
```

Ici, on avait préparé le site Web au préalable. On met alors nos fichiers sur ***/var/www/html***

Pour continuer, on va modifier le fichier de configuration qu'utilise Nginx pour qu'il récupère les fichiers du site dans ***/var/www/html***

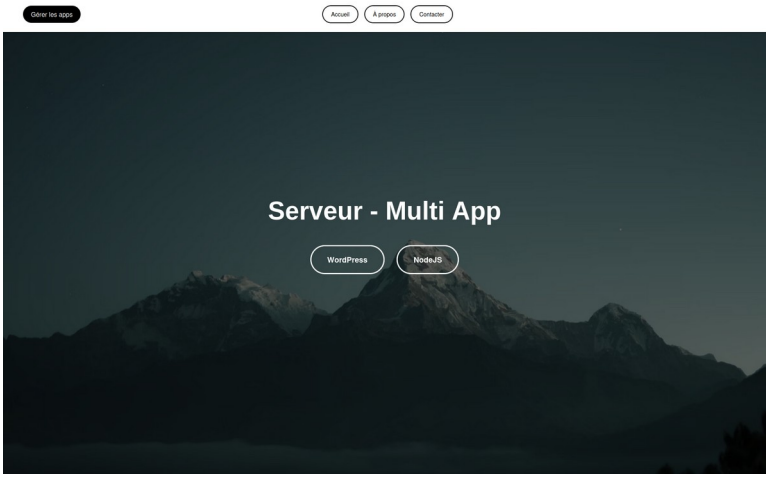



```
server {
    listen 80;
    server_name vm-roueur1.tail66b3dd.ts.net;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name vm-roueur1.tail66b3dd.ts.net;

    ssl_certificate /etc/ssl/certs/serveur.crt;
    ssl_certificate_key /etc/ssl/private/serveur.key;

    root /var/www/html;

    index index.html index.php;

    location / {
        try_files $uri $uri/ =404;
    }

    location /wordpress {
        try_files $uri $uri/ /wordpress/index.php?
$args;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;

        fastcgi_pass unix:/var/run/php/php8.4-
fpm.sock;
    }

    location /nodejs_app/ {
        proxy_pass http://localhost:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;

        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

(Le fichier **.conf** *possède déjà le nom de domaine donné par TailScale, WordPress & NodeJS car on a effectué la documentation après avoir réalisé le projet.)*

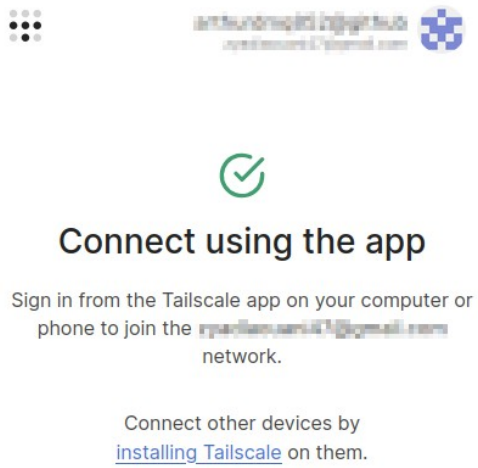
Une fois que la configuration a été verifiée avec ***sudo nginx -t***, on peut relancer Nginx (***sudo systemctl restart nginx***)

Si tout marche correctement, on arrive sur notre site avec ***localhost*** dans le navigateur.



Afin que l'on puisse se connecter depuis un autre ordinateur qui n'est pas sur le même réseau, on utilise ici Tailscale.

On l'installe avec le guide présent sur le site officiel de Tailscale. Si tout fonctionne correctement sur le serveur, sur un autre PC qui possède également Tailscale, on récupére le lien de la machine serveur.



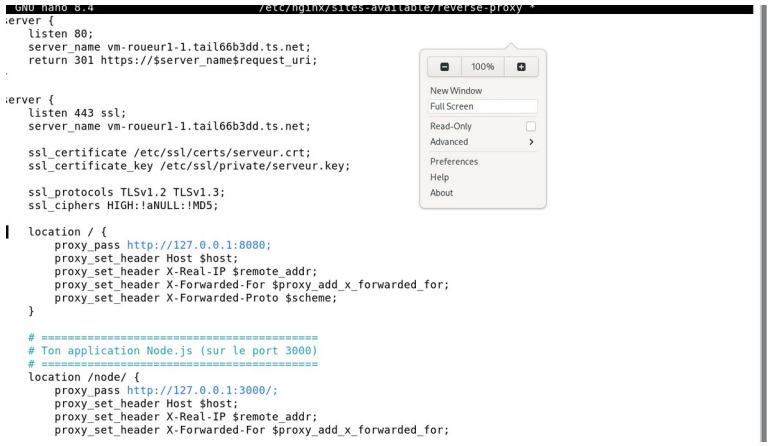
Une fois que les deux machines sont connectées entre elles, lorsque l'on se connecte avec l'IP donné par (ici **100.65.93.87**), on obtient la page du site.

L'ajout & le rôle du Reverse Proxy

Le reverse proxy a été mis en place avec Nginx pour centraliser et sécuriser l’accès aux applications du serveur.

Il redirige automatiquement le trafic HTTP vers le HTTPS, puis transmet les requêtes vers les services internes.

La page principale est envoyée vers un serveur web Nginx local (port 8080), tandis que l’application Node.js est accessible via l'alias **/nodejs_app/** (port 3000).



sudo nano /etc/nginx/sites-available/reverse-proxy

sudo nginx -t

sudo systemctl reload nginx

```
oot@vm-roueur1:~# nginx -t
ginx: the configuration file /etc/nginx/nginx.conf syntax is ok
ginx: configuration file /etc/nginx/nginx.conf test is successful
oot@vm-roueur1:~#
```

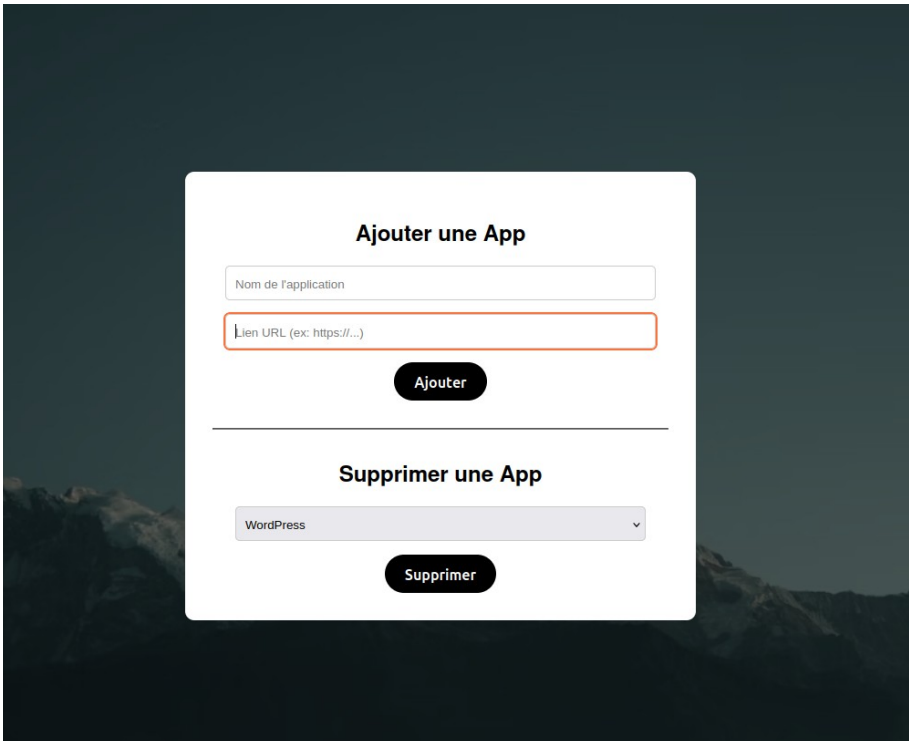
Pour valider la mise en place, la syntaxe du fichier a été testée avec la commande ***sudo nginx -t*** pour éviter toute interruption de service, puis Nginx a été rechargé.

Le bon fonctionnement est confirmé par l'accès aux différentes applications depuis l'extérieur avec une connexion sécurisée par certificat TLS.

Comment marche l'interface de gestion ?

Pour ajouter une application, il faut d'abord l'ajouter dans le dossier du serveur ***/var/www/html***. Une fois cela fait, on l'ajoute sur le site web via notre page faite pour.

On peut alors écrire le nom de notre application et indiquer le chemin dans le dossier



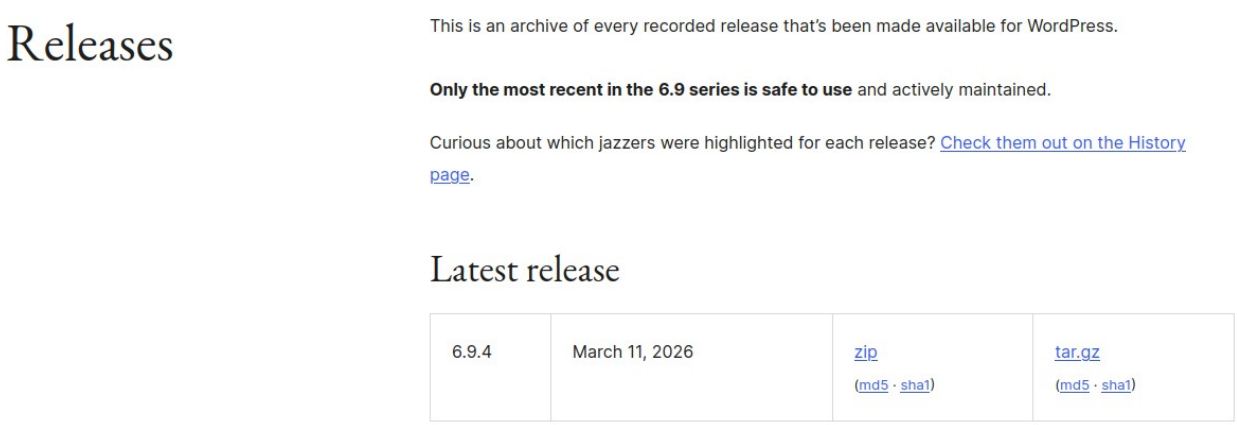
Pour supprimer une application, il suffit de la sélectionner dans la liste, et de cliquer sur supprimer.

Exemple d'ajout d'applications

Pour tester notre gestion d'applications, on va ajouter WordPress & NodeJS sur notre site.

WordPress

Afin d'ajouter WordPress, on commence par télécharger sur leur site

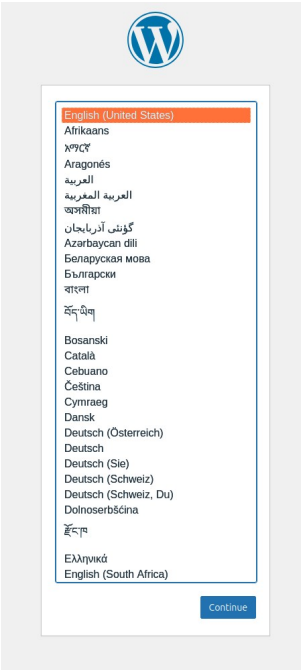


On l'extrait ensuite dans ***/var/www/html*** et on lui donne les permissions

sudo chown -R www-data:www-data /var/www/html/wordpress
sudo chmod -R 755 /var/www/html/wordpress

Puis on l'ajoute sur notre site en mettant le chemin de wordpress

Si tout marche correctement, lorsque l'on clique sur le bouton WordPress sur la page d'accueil de notre site, on arrive sur la page de configuration de WordPress.



NodeJS

Pour ajouter NodeJS à notre serveur Web, on commence par se rendre dans notre fichier web

sudo nano /etc/nginx/sites-available/reverse-proxy

Et on ajoute ces lignes de commandes pour que notre site détecte NodeJS

```
location /nodejs_app {
proxy_pass http://localhost:3000;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
```

Une fois cela fait, on crée un raccourci dans le dossier des sites activés avec

sudo ln -s /etc/nginx/sites-available/reverse-proxy /etc/nginx/sites-enabled/

Et on vérifie que la syntax est bonne puis on redémarre nginx avec

```
sudo nginx -t
sudo systemctl restart nginx
```

Maintenant que ces étapes sont faites, on peut réellement ajouter NodeJS.

Pour ça, on va crée un fichier ***app.js*** dans le dossier de notre site avec

```
cd /var/www/html/nodejs_app
sudo nano app.js
```

Et on écrit ces lignes de commandes dedans :

```
const http = require('http');
const server = http.createServer((req, res) => {
res.writeHead(200, {'Content-Type': 'text/plain'});
res.end('Node.js is ALIVE on port 3000!\n');
});
server.listen(3000, '127.0.0.1');
```

Enfin, on lance PM2 (Le gestionnaire de processus de NodeJS)

```
pm2 start app.js --name "node-app"
pm2 save
```

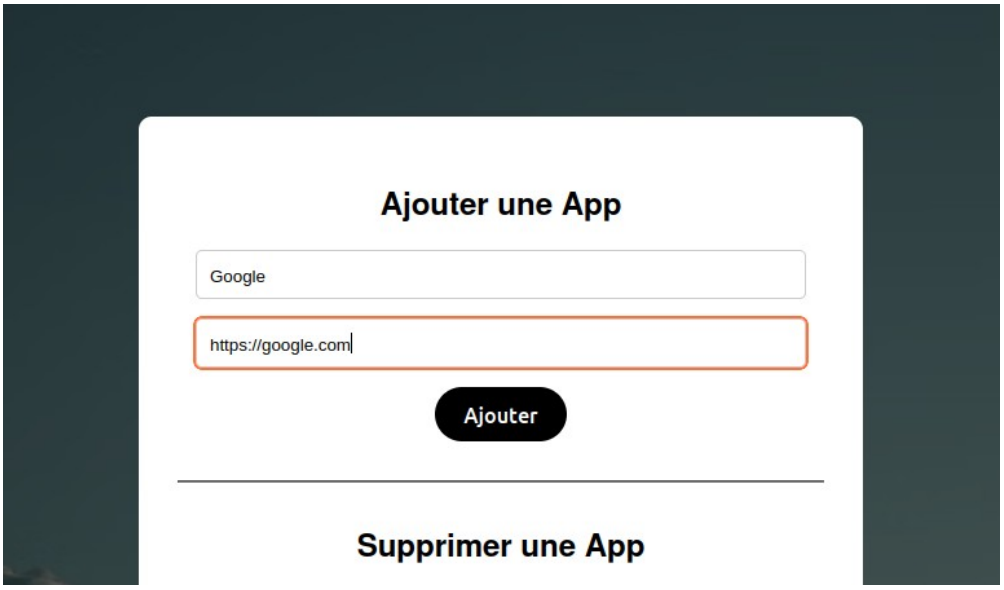
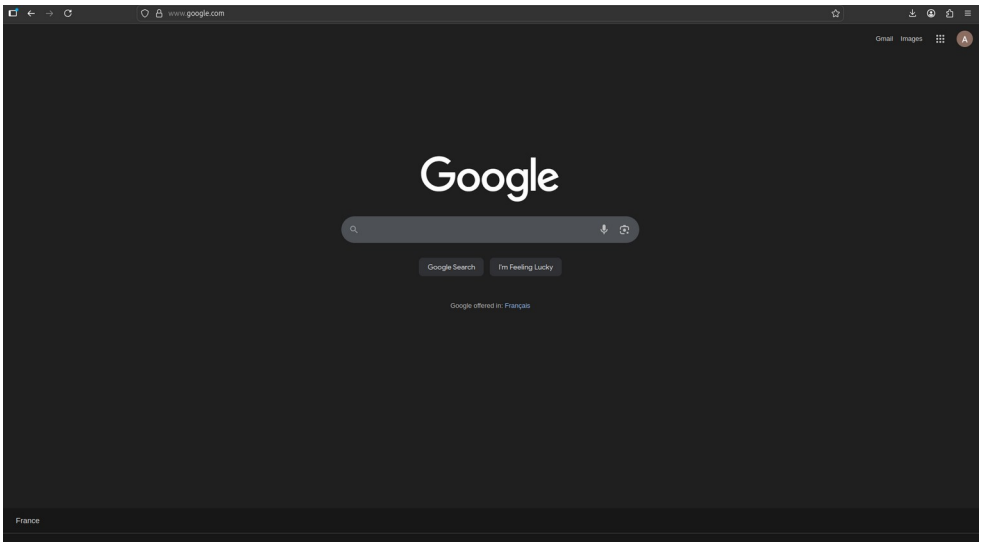
Et on vérifie que notre application soit bien lancée avec

curl http://localhost:3000
Si on a correctement un message nous indiquant ***Node.js is ALIVE***

On peut ajouter NodeJS sur notre site avec le lien ***/nodejs_app/***, et on clique sur le bouton à l'accueil.

```
Node.js is ALIVE on port 3000!
```

Bonus

Notre site permet de rediriger vers des liens web.

Pour cela, lorsque l'on souhaite ajouter une application à notre site, on y ajoute le nom de notre site, et le lien
(Attention à bien ajouter ***https://*** afin que le site détecte que ce soit un site extérieur, et non pas un site ajouté dans ses dossiers.)